

Shipay Back-End Engineer Challenge

Questão 1 — Validação de cadastro

Proposta

A solução será implementada como uma API síncrona, responsável por validar um cadastro a partir de um CNPJ e um CEP.

A entrada será validada inicialmente utilizando Pydantic, garantindo que os dados recebidos estejam em formato válido antes de qualquer chamada externa.

Como as consultas de CNPJ e CEP são independentes, elas poderão ser executadas de forma concorrente internamente. Apesar disso, a API continuará sendo síncrona para o cliente, que receberá a resposta somente após a conclusão da validação.

Arquitetura

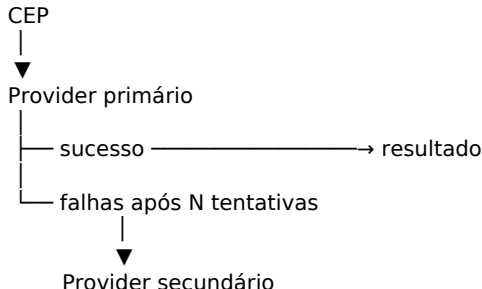
A aplicação será organizada em torno de um ValidationService, responsável pela regra de negócio e pela comparação dos endereços.

O serviço dependerá de duas abstrações: CnpjProvider e CepResolver.

O CnpjProvider será responsável pela comunicação com o serviço externo de CNPJ, encapsulando timeout, retry para falhas transitórias e backoff entre tentativas.

Para CEP, será utilizado um CepResolver, pois existem dois providers disponíveis. O resolver será responsável exclusivamente pela estratégia de primary → fallback.

Fluxo do CEP



Adapters

Como os providers possuem contratos diferentes, cada integração será encapsulada por um Adapter.

BrasilAPI:
uf
municipio
logradouro

ViaCEP:
uf
localidade
logradouro

Os adapters convertem essas respostas para um modelo interno comum:

Address
├─ state
├─ city
└─ street

Dessa forma, o ValidationService não precisa conhecer detalhes específicos de nenhum fornecedor.

Provider Factory

A criação dos providers será centralizada em uma `ProviderFactory`. A `Factory` receberá configurações como URL dos providers, timeout, quantidade máxima de retries e configuração de primary/fallback.

Ela será responsável pela criação das implementações concretas dos providers. Assim, o restante da aplicação trabalha contra abstrações, e uma mudança de fornecedor fica isolada na composição da aplicação e em seu respectivo `Adapter`, quando necessário.

Normalização e comparação

Após as consultas, os dados serão normalizados antes da comparação: remoção de espaços desnecessários, comparação sem distinção entre maiúsculas e minúsculas e normalização de acentuação.

A comparação será realizada sobre UF, cidade e logradouro.

Se os dados forem compatíveis, a API retornará HTTP 200. Caso os dados sejam válidos, mas apresentem divergência entre CNPJ e CEP, a API retornará HTTP 404, conforme especificado no desafio.

Falhas que impeçam a obtenção dos dados dos providers, mesmo após retry e fallback, não serão tratadas como divergência de endereço. Nesse caso, será retornado um erro de dependência/servidor, como HTTP 5xx/503.

Resiliência

Os retries serão limitados e aplicados somente a falhas consideradas transitórias, como timeouts, erros de conexão e determinadas respostas 5xx.

Não serão realizadas tentativas adicionais para erros que indiquem uma falha permanente da requisição, como dados inválidos.

Nos providers de CEP, o circuit breaker será utilizado para evitar chamadas repetidas a uma dependência que esteja apresentando indisponibilidade persistente. Cada provider terá seu próprio circuito, permitindo que o `CepResolver` utilize o provider alternativo quando o primário estiver indisponível.

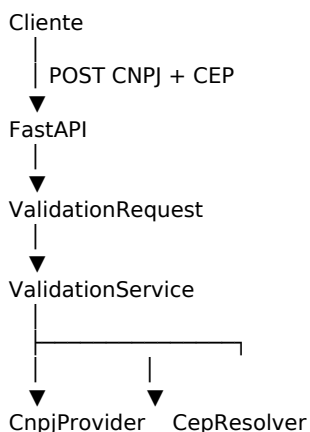
CLOSED → OPEN → HALF-OPEN → CLOSED

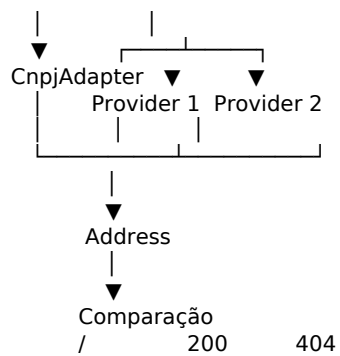
Latência

Como a API precisa permanecer síncrona, o tempo total da requisição será limitado por um orçamento de latência definido a partir da latência esperada dos providers, timeout configurado, quantidade máxima de retries, backoff, processamento interno e margem de segurança.

A execução concorrente das consultas independentes de CNPJ e CEP reduz a latência do caminho crítico, enquanto os timeouts e retries limitados impedem que uma dependência externa mantenha a requisição aberta indefinidamente.

Fluxo resumido





Documento atualizado conforme a arquitetura final da Questão 1.